EVOLVE

구조 진화와 엔지니어링 운영

코드 분포를 설계 결론으로 오인하지 않고 불변조건·가변성·검증 순서로 리팩터링한다

> **구조 개선은 파일을 작게 만드는 작업이 아니라, 실패했을 때 책임을 판정할 수 있도록 의미의 소유권을 다시 배치하는 작업이다.**

대상 snapshot
v2.1.88 / 2ca5ddabfed5

설명 구조
조건 · 이론 · 구현 · 반례

제3자 복원 소스에 대한 교육용 분석이다. 코드 관찰, 이론적 비유, 개선 제안을 구분하며 누락 모듈을 추정해 채우지 않는다.

큰 파일은 원인이 아니라 증상일 수 있다

어떤 결합이 실제 구조 문제인가?

성립 조건

QueryEngine/query/REPL/Tool/sessionStorage에 여러 변화 이유가 겹친다.

이론 렌즈

cohesion은 함께 변해야 하는 것의 결속이고 coupling은 독립 변화가 서로를 흔드는 정도다.

코드의 구체화

turn lifecycle, UI, provider, effect, persistence가 동일 type/file boundary를 가로지른다.

깨지는 지점

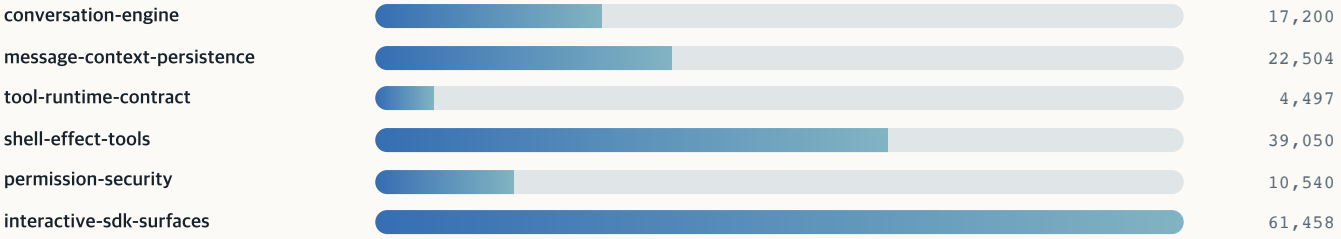
줄 수 기준으로 자르면 protocol invariant가 여러 module에 복제된다.

해석 분해 단위는 코드 양이 아니라 state/effect/invariant ownership이다.

근거와 연결 src/QueryEngine.ts · src/Tool.ts · src/utils/sessionStorage.ts

분포는 설계 질문을 고르는 도구다

> 어떤 책임 영역에 어떤 압력이 모이는가?



line 분포는 importance rank가 아니다. shell 영역에서는 security 질문을, interactive 영역에서는 reliability/concurrency 질문을, persistence 영역에서는 performance/recovery 질문을 우선 탐색하게 하는 research prior다.

근거와 연결 generated metrics

언어 메커니즘의 희소성과 집중을 함께 본다

> generator가 핵심이라는 말은 수치로 어떻게 교정되는가?

영역	async fn	async gen	yield	await
conversation-engine	96	11	83	162
message-context-persistence	178	1	1	242
tool-runtime-contract	17	9	40	18
interactive-sdk-surfaces	264	2	7	754
extension-adapters	373	15	49	772

전체 async generator는 50개이고 await는 5,238개다. 희소한 generator가 turn protocol에서 결정적일 수 있지만, 비동기 correctness의 대부분은 일반 await 경계와 cancellation/cleanup에 분포한다.

주석의 문제 언어를 architecture hypothesis로 바꾼다

> Without this, previously, race 같은 표현은 무엇을 알려 주는가?

영역	reliability	performance	security	compatibility	concurrency
conversation-engine	273	186	19	36	42
message-context-persistence	203	276	50	35	36
shell-effect-tools	160	125	699	142	29
interactive-sdk-surfaces	631	349	159	49	176
extension-adapters	342	403	115	80	74

이는 lexical marker를 포함한 comment line 수다. incident 수나 품질 점수가 아니며, 어떤 quality scenario를 먼저 작성할지 선택하는 단서다.

분포를 검증 가능한 scenario로 내린다

> metric에서 architecture decision으로 바로 점프하지 않으려면?

관찰 신호	scenario	response measure 후보
FileEdit race 주석	외부 변경이 validation 직후 발생	stale overwrite 0
fallback branch	stream 뒤 provider 오류	duplicate tool effect 0
REPL generation guard	cancel 직후 resubmit	new prompt state 손실 0
cache comments	tool/context 순서 변화	semantic-equivalent prefix 안정
session repair	kill 후 resume	orphan pair 0, unknown 보존

근거와 연결 artifacts/knowledge/architecture/architecture-characteristics-and-tradeoffs.md

추출의 완료 조건은 call site 수가 아니다

> 새 engine이 무엇을 소유해야 migration이 끝나는가?

책임	현재 신호	완료 oracle
turn admission	submit/process input	모든 entrypoint 동일 trace
context build	session + query context	provider request equivalence
terminal settlement	query return handling	terminal reason 완전성
durability ordering	transcript write	kill/resume corpus
surface projection	REPL/SDK adapters	허용 차이 명시

migration은 REPL 코드를 복사해 class로 넣는 일이 아니라 last writer와 invariant owner를 하나씩 이동하는 과정이다.

근거와 연결 src/QueryEngine.ts · src/screens/REPL.tsx

facet 분해는 계약을 약하게 만들 위험도 있다

> 어떤 seam부터 만들어야 하는가?

01	**compat adapter**	기존 Tool을 새 facet view로 투영
02	**effect facet**	validate/call/result truth
03	**authority facet**	permission request와 decision evidence
04	**model facet**	name/description/schema/cache identity
05	**presentation facet**	REPL render/search/progress
06	**fitness**	effect↔React, presenter↔call 의존 금지

한 번에 interface를 교체하지 않고 기존 contract snapshot과 event trace로 facet 사이 의미 손실을 탐지한다.

journal, reader, repair, projection은 다른 시간 방향을 가진다

왜 하나의 service에서 분리할 가치가 있는가?

unit	시간 방향	핵심 contract
writer	현재→future	append identity/order/flush
reader	past→current	versioned decoding
repair	invalid past→admissible past	명시 rule + loss record
projector	history→consumer view	deterministic/loss budget
compactor	history→summary entity	provenance/omission

분해 기준 함수 종류가 아니라 서로 다른 failure semantics와 compatibility owner를 기준으로 한다.

feature gate는 if문보다 넓은 제품군 문제다

왜 dynamic import와 excluded symbol 검사가 필요한가?

성립 조건

provider/build/cohort에 따라 포함 가능한 code와 capability가 달라진다.

이론 렌즈

Software Product Line은 commonality와 variability, feature constraint를 모델링한다.

코드의 구체화

feature() call, mode filtering, dynamic module loading이 artifact/runtime variant를 만든다.

깨지는 지점

gate 밖 static import나 문자열이 build artifact에 남으면 runtime false여도 product constraint를 위반한다.

해석 feature variability는 code path뿐 아니라 module graph와 produced artifact를 acceptance 대상으로 만든다.

근거와 연결 SEI FODA · src/tools.ts

모든 gate 조합을 시험할 수 없다

> 조합 폭발을 어떻게 줄이되 위험을 숨기지 않는가?

축	대표 값	상호작용 위험
entrypoint	REPL/SDK/bridge	projection 차이
provider	direct/Bedrock/Vertex	header/cache/tool support
feature	on/off/cohort	module/artifact inclusion
platform	POSIX/Windows	shell/path/mtime
failure	success/abort/fallback	repair/duplicate effect

pairwise selection은 coverage 도구일 뿐 safety-critical interaction을 대체하지 않는다. known high-risk triples는 별도 scenario로 고정한다.

불완전 복원본에서는 기존 의미를 먼저 포획한다

unit test를 더 쓰는 것만으로 왜 부족한가?

성립 조건

완전한 test suite/build module과 missing type source가 없다.

이론 렌즈

characterization은 현재 observable behavior를 migration oracle로 만든다.

코드의 구체화

CoreEvent sequence, effect diff, terminal, journal/resume, request hash를 fixture별로 기록한다.

깨지는 지점

현재 bug까지 무비판적으로 영구화하거나 observer가 behavior를 바꾸면 oracle이 오염된다.

해석	characterization은 정답 선언이 아니라 의도적으로 바꿀 차이와 보존할 차이를 분리하는 baseline이다.

telemetry는 활동량이 아니라 인과 join을 제공해야 한다

> logEvent가 많다는 사실만으로 무엇이 부족한가?

identity	join	결정 지원
query chain	parent/subturn	recursive work 원인
request attempt	fallback/provider	실패 funnel
tool use	proposal/effect/result	partial/duplicate effect
session/message	runtime/durable	resume drift
feature cohort	variant/result	rollout 비교

event count는 acceptance가 아니다. exact input→decision→effect→outcome join이 없으면 최적화 주장은 보류한다.

근거와 연결 Engineering Operating Model

위험이 높은 의미부터 owner를 고정한다

어떤 순서가 문제 원인을 분리하는가?

01	01 trace baseline	현재 turn/effect/durability 의미를 고정
02	02 conversation owner	QueryEngine으로 admission·terminal·ordering 통합
03	03 projection adapters	REPL/SDK 차이를 명시
04	04 Tool facets	model/authority/effect/presentation 분리
05	05 journal model	writer/reader/repair/projector 분리
06	06 variability fitness	variant별 module/artifact 검사

순서의 이유 동시에 여러 축을 바꾸면 semantic drift의 owner와 rollback 지점을 찾을 수 없다.

좋은 분리는 latency와 comprehensibility를 악화시킬 수 있다

> 구조 결정의 sensitivity point는 어디인가?

tactic	강화	약화 가능성	sensitivity
더 많은 validation layer	safety/diagnosis	latency/UX	tool call critical path
세분화된 facet	replaceability/testability	adapter 수/trace 복잡도	cross-facet invariant
더 durable한 기록	recoverability	I/O latency/privacy	pre-query flush
강한 isolation	security	capability/usability	shell/tool coverage
더 많은 feature gate	rollout safety	combination complexity	persistent format

Tier 0/1 safety·correctness를 평균 점수로 희생하지 않고, 그 feasible set 안에서 Tier 2/3 비용을 비교한다.

문서의 경계를 CI 실패로 바꾼다

> 새 structure가 다시 무너지지 않게 하는 최소 규칙은?

rule	evidence
engine core에서 React/Ink import 금지	dependency fitness
provider payload는 ContextIR assembler만 생성	construction-site scan
ToolEffect에서 Presenter import 금지	boundary map
journal record는 versioned union	schema + historical corpus
feature excluded symbol/string 검사	build artifact scan
CoreEvent projection exhaustive	compile-time + fixture

완료 조건 디렉터리가 그럴듯한 것이 아니라 잘못된 dependency와 누락된 variant가 자동으로 실패해야 한다.

리팩터링은 배포 뒤 feedback까지 포함한다

코드 merge가 끝이 아닌 이유는 무엇인가?



근거와 연결 Engineering Operating Model

용어 암기가 아니라 설계 판단을 평가한다

> 학습자가 무엇을 만들어야 하는가?

수준	과제	통과 기준
mechanism	async generator state를 설명	return/throw/cancel 비대칭 포함
invariant	tool pairing 반례 구성	fallback/abort/resume 모두 설명
effect	Edit race 재현	TOCTOU window와 limitation 명시
architecture	Tool facet proposal	adapter/oracle/rollback 포함
operation	rollout scenario 작성	identity join과 unknown 보존
critique	보고서 비유 반박	코드·1차 자료로 경계 수정

진화 가능한 구조는 의미의 이동을 추적할 수 있다

이 권에서 남겨야 할 한 문장은 무엇인가?

> **리팩터링의 단위는 파일이 아니라 invariant ownership이고, 완료의 증거는 diff가 아니라 실패 scenario에서의 동등성이다.**

분포와 주석은 어디를 볼지 알려 주고, 이론은 어떤 반례를 만들지 알려 준다. 최종 구조 판단은 고정된 코드 관찰·quality scenario·검증 가능한 migration seam의 결합에서 나온다.